

Recherche de documents « *Document Retrieval* »

Séance 4

Représentation vectorielle des documents

I. Introduction :

Après avoir préparé notre corpus, nettoyé les textes et extrait des unités lexicales normalisées (tokens, lemmes, racines, etc.), nous sommes prêts à passer à l'étape suivante : représenter les documents sous forme numérique. Cette transformation est essentielle, elle permet aux algorithmes de comparer, classer et rechercher des documents de manière efficace.

La représentation vectorielle consiste à convertir un texte en vecteur de nombres, où chaque dimension correspond à un aspect du contenu lexical ou statistique. Chaque dimension peut représenter un terme, une fréquence, ou une statistique dérivée du texte. Cette étape est à

la base de nombreuses méthodes de recherche d'information et de traitement automatique du langage. Avant l'avènement des modèles neuronaux profonds, les méthodes classiques reposaient sur des approches statistiques simples et interprétables, toujours largement utilisées en pratique pour leur efficacité et leur faible coût computationnel.

Après les étapes de prétraitement, la vectorisation constitue le pont entre les textes bruts et les modèles d'analyse. Lors de cette séance, vous allez :

- Comprendre les principes fondamentaux qui permettent de passer d'un texte brut à une représentation vectorielle ;
- Implémenter des méthodes classiques de vectorisation sans utiliser de bibliothèques externes comme *NLTK*, *Scikit-learn* ou *Gensim* ;
- Expérimenter ces représentations pour comparer et analyser des documents, afin de comprendre leur fonctionnement et leurs limites.

Bien que ces représentations vectorielles soient simples, rapides et efficaces, elles présentent aussi des limites : elles ne capturent que partiellement la structure du langage, l'ordre des mots étant souvent réduit ou simplifié (même si des techniques comme les n-grammes permettent d'en conserver une partie). Elles ignorent également la polysémie, la synonymie et les relations sémantiques plus profondes, ce qui motive l'apparition de méthodes plus avancées. Cette séance vous donnera une vision concrète de la manière dont les textes deviennent exploitables par des algorithmes, tout en posant les bases pour des techniques plus avancées qui intègrent le contexte et la sémantique.

II. Plan de la quatrième étape de l'atelier :

1. Représentations vectorielles simples

Avant d'exploiter des modèles statistiques, il est essentiel de comprendre comment un corpus textuel peut être converti en structures de données manipulables par un ordinateur.

Nous allons donc explorer les premières méthodes pour transformer un texte en vecteur numérique sans utiliser de statistiques sur le corpus. Ces représentations simples permettent de manipuler directement les mots ou les phrases grâce à des vecteurs exploitables par des algorithmes, marquant ainsi le passage du texte brut (symbolique) à une représentation numérique utilisable pour la recherche, la classification ou la mesure de similarité.

Exemple : Soit un mini-corpus de 4 phrases.

- "chat dort"
- "chien dort"
- "chat mange"
- "chien mange maison"

Vocabulaires (après tokenisation et filtrage, ordre arbitraire) :

- `vocabulaire = ['chat', 'chien', 'dort', 'mange', 'maison']`
- `vocabulaire_direct = {'chat':0, 'chien':1, 'dort':2, 'mange':3, 'maison':4}`
- `vocabulaire_inverse = {0:'chat', 1:'chien', 2:'dort', 3:'mange', 4:'maison'}`

a. Représentation par indices

Lors de la séance précédente, vous avez construit :

- Un vocabulaire filtré du corpus.
- Deux dictionnaires d'index :
 - `vocabulaire_direct: {mot : index}`
 - `vocabulaire_inverse: {index : mot}`

Ces dictionnaires permettent de représenter chaque mot par un entier unique. Dans cette sous-section, vous allez utiliser ces dictionnaires pour encoder une phrase en indices et décoder une séquence d'indices en texte.

Exemples :

- Phrase "chat dort" → `[0, 2]`
- Phrase "chien mange maison" → `[1, 3, 4]`
- Phrase "souris mange fromage" → `[-1, 3, -1]`
- Le vecteur `[1,2,0,3]` → "chien dort chat mange"
- Le vecteur `[10,2,3]` → "<UNK> dort mange"
- Écrire une fonction **`texte_vers_indices(texte, vocabulaire_direct)`** qui transforme une phrase ou un document en une liste d'indices correspondant aux mots dans le vocabulaire. Elle reçoit une phrase ou un document ainsi que le `vocabulaire_direct` construit à partir d'un corpus selon une des configurations de la séance précédente ; elle retourne une liste d'entiers correspondant aux indices des mots présents dans le texte. Les mots absents du vocabulaire peuvent être ignorés ou mappés à un index spécial (`-1` ou `None` selon votre choix).

- Écrire une fonction **indices_vers_texte(indices,vocabulaire_inverse)** qui transforme une liste d'indices en un texte. Elle reçoit une liste d'entiers représentant des mots ainsi que le vocabulaire_inverse ; elle retourne une chaîne de caractères correspondant au texte reconstruit. Les indices invalides ou hors dictionnaire peuvent être ignorés ou remplacés par un token spécial (<UNK>).

b. One-Hot Encoding

Le One-Hot Encoding est une représentation vectorielle où chaque mot du vocabulaire est codé par un vecteur de taille $|\text{Vocabulaire}|$:

- La position correspondant au mot vaut 1, toutes les autres positions valent 0.
- Cette représentation est orthogonale et unique pour chaque mot.
- Elle est utilisée comme vecteur d'entrée dans des architectures comme Word2Vec, où le vecteur final sera ensuite appris comme embedding dense.

Exemple :

Mot	One-Hot
"chat"	[1,0,0,0,0]
"chien"	[0,1,0,0,0]
"dort"	[0,0,1,0,0]
"mange"	[0,0,0,1,0]
"maison"	[0,0,0,0,1]

Ainsi, le texte "chat maison" devient la liste de vecteurs : $[[1,0,0,0,0], [0,0,0,0,1]]$.

Cette représentation conserve l'ordre des mots dans le texte, ce qui permet de reconstruire la phrase dans son ordre original, contrairement au Sac-de-mots (BoW) présenté ensuite.

- Écrire une fonction **one_hot_encode_mots(texte, vocabulaire_direct)** qui transforme un texte en une liste de vecteurs One-Hot par mot. Elle retourne une liste de vecteurs One-Hot (un vecteur par mot).
- Écrire une fonction **one_hot_decode(vecteurs, vocabulaire_inverse)** qui transforme une liste de vecteurs One-Hot en texte. Elle retourne une chaîne de caractères correspondant au texte reconstruit.

c. Sac de mots (Bag-of-Words «BoW») binaire

Les représentations mot par mot, comme la représentation des indices ou le One-Hot Encoding, codent chaque mot individuellement. Pour un texte ou un document :

- Chaque phrase devient une séquence de vecteurs de longueur variable (en nombre de vecteurs correspondant aux mots).

- Deux documents différents ont rarement la même longueur.
- Il est donc impossible de comparer directement deux documents avec une distance classique (Euclidienne, cosinus...) à partir de ces séquences.

Pour résoudre ce problème, on utilise le Sac-de-mots (Bag-of-Words) binaire :

- Le modèle du Sac de mots consiste à représenter un document par la liste de ses mots sans tenir compte de l'ordre, chaque mot correspondant à une dimension du vecteur.
- Chaque document est représenté par un vecteur de longueur fixe = taille du vocabulaire.
- Chaque dimension vaut 1 si le mot correspondant apparaît au moins une fois dans le document, 0 sinon.
- Cette représentation permet de comparer facilement des documents en utilisant des mesures de similarité ou des distances.

Exemple :

Document	chat	chien	dort	mange	maison
"chat dort"	1	0	1	0	0
"chien dort"	0	1	1	0	0
"chat mange"	1	0	0	1	0
"chien mange maison"	0	1	0	1	1

Chaque dimension indique la présence ou l'absence d'un mot du vocabulaire.

- Écrire une fonction **bag_of_words_binaire(texte, vocabulaire_direct)** qui transforme un texte en un vecteur binaire représentant la présence des mots. Elle retourne un vecteur binaire (liste d'entiers 0/1) de taille |vocabulaire|. Les mots absents sont ignorés.
- Écrire une fonction **bag_of_words_decode(vecteur, vocabulaire_inverse)** qui transforme un vecteur binaire en texte (liste des mots présents). Elle retourne une chaîne de caractères correspondant aux mots présents.

Bien que cette représentation permette de comparer facilement des documents, elle ne tient pas compte de la fréquence des mots ni de leur importance. Nous verrons ces aspects dans la prochaine étape avec les représentations pondérées.

2. Représentations pondérées et statistiques

Dans la section précédente, nous avons appris à représenter les mots et les documents de manière simple, avec des vecteurs binaires ou des One-Hot Encoding. Ces méthodes sont faciles à comprendre, mais elles ignorent l'importance relative des mots et les différences de longueur entre documents. Pour améliorer la représentation des documents et préparer la recherche d'information, nous allons maintenant explorer des représentations pondérées :

- Bag-of-Words occurrences : compter combien de fois chaque mot apparaît dans un document.
- TF (Term Frequency) : normaliser la fréquence par la longueur du document.
- IDF (Inverse Document Frequency) : mesurer l'importance d'un mot à travers tout le corpus.
- TF-IDF : combiner TF et IDF pour pondérer chaque mot selon sa fréquence et son importance.
- BM25 : modèle probabiliste amélioré pour la recherche d'information, basé sur TF-IDF avec paramètres pour ajuster l'impact des fréquences et de la longueur des documents.

a. Sac-de-mots (Bag-of-Words «BoW») occurrences

Le Bag-of-Words binaire indique uniquement si un mot est présent ou non. Une première amélioration consiste à compter combien de fois chaque mot apparaît dans le document. Cela donne un vecteur de comptage, plus riche que la simple présence/absence.

Exemple :

Document	chat	chien	dort	mange	maison
"chat dort"	1	0	1	0	0
"chien dort"	0	1	1	0	0
"chat mange"	1	0	0	1	0
"chien mange maison"	0	1	0	1	1

Idem que le BoW binaire ici car tous les mots apparaissent une seule fois.

- Écrire une fonction **bag_of_words_occurrences(texte,vocabulaire_direct)** qui transforme un texte en un vecteur de comptage des mots. Elle retourne un vecteur d'entiers (liste) représentant le nombre d'occurrences de chaque mot du vocabulaire.
- Écrire une fonction **bag_of_words_occurrences_decode(vecteur, vocabulaire_inverse)** qui transforme un vecteur de comptage en texte. Elle retourne une chaîne de caractères correspondant aux mots présents (répétés selon leur fréquence).

b. TF (Term Frequency)

Avec le Bag-of-Words occurrences, chaque document est représenté par un vecteur indiquant le nombre de fois qu'un mot apparaît. Ces fréquences brutes favorisent toutefois les documents longs, car les mots ont plus de chances d'apparaître simplement du fait de la longueur du texte. Pour corriger ce biais et permettre une comparaison équitable entre documents de tailles différentes, on normalise chaque fréquence par le nombre total de mots dans le document. Cette normalisation produit la Term Frequency (TF) :

$$TF(t, d) = \frac{\text{nombre d'occurrences du mot } t \text{ dans le document } d}{\text{nombre total de mots dans } d} = \frac{f_{t,d}}{\sum_w f_{w,d}}$$

où $f_{t,d}$ est la fréquence brute du mot t dans le document d .

Les valeurs sont comprises entre 0 et 1. Le TF représente donc la proportion de chaque mot dans le document, plutôt que sa fréquence brute.

Exemple :

Document	chat	chien	dort	mange	maison	Total mots	TF
"chat dort"	1	0	1	0	0	2	[0.5,0,0.5,0,0]
"chien dort"	0	1	1	0	0	2	[0,0.5,0.5,0,0]
"chat mange"	1	0	0	1	0	2	[0.5,0,0,0.5,0]
"chien mange maison"	0	1	0	1	1	3	[0,0.33,0,0.33,0.33]

- Écrire une fonction **calcul_tf(texte, vocabulaire_direct)** qui transforme un texte en un vecteur TF (Term Frequency). Elle retourne un vecteur de « float » représentant la fréquence relative de chaque mot dans le document. Grâce au TF, la représentation exprime « l'importance relative du mot dans le document », et non juste son nombre brut d'occurrences.

c. IDF (Inverse Document Frequency)

Même après le nettoyage du corpus (suppression des stopwords, normalisation, filtrage des tokens trop courts ou trop longs), certains mots restent très fréquents dans une grande partie du corpus, tandis que d'autres apparaissent seulement dans quelques documents. Ces mots fréquents apportent peu d'information pour distinguer les documents entre eux.

Pour tenir compte de cela, on utilise IDF (Inverse Document Frequency), qui mesure l'importance d'un mot à l'échelle du corpus entier :

- un mot rare dans le corpus doit avoir plus de poids ;
- un mot présent dans presque tous les documents doit avoir moins de poids.

L'IDF dépend de la fréquence documentaire « df_t », c'est-à-dire du nombre de documents contenant le mot « t ». Il existe plusieurs variantes de l'IDF, utilisées selon les besoins. Les principales sont présentées ci-dessous.

i. Variante « classique »

C'est la forme la plus simple et la plus répandue :

$$IDF_classique(t) = \log \frac{N}{df_t}$$

- N : nombre total de document.
- df_t : nombre de documents contenant le mot t

Un mot apparaissant dans peu de documents obtient un IDF élevé, tandis qu'un mot présent dans la majorité des documents obtient un IDF faible. Le problème est que si un mot apparaît dans tous les documents, $IDF = \log(1) = 0 \Rightarrow$ **perte totale de poids.**

Exemple :

N = 4 documents

df (nombre de documents contenant le mot)	IDF de chaque mot
chat → 2	chat : $\log(4/2) = \log(2) \approx 0.693$
chien → 2	chien : $\log(4/2) = 0.693$
dort → 2	dort : $\log(4/2) = 0.693$
mange → 2	mange : $\log(4/2) = 0.693$
maison → 1	maison : $\log(4/1) = \log(4) \approx 1.386$

Le vecteur IDF = [0.693, 0.693, 0.693, 0.693, 1.386].

La même taille que le vocabulaire.

ii. Variante « smooth » (IDF lissée)

Permet d'éviter les divisions extrêmes et stabilise les valeurs :

$$IDF_smooth(t) = \log\left(\frac{N}{df_t + 1}\right)$$

Ajouter +1 à l'extérieur du log pour éviter les valeurs nulles :

$$IDF_smooth_P1(t) = \log\left(\frac{N}{df_t + 1}\right) + 1$$

Cette formule empêche d'obtenir des valeurs nulles ou négatives et est très utilisée dans les implémentations modernes.

iii. Variante « logplus »

Elle ajoute 1 dans le logarithme afin d'amplifier légèrement les différences entre mots fréquents et moins fréquents :

$$IDF_logplus(t) = \log\left(1 + \frac{N}{df_t}\right)$$

Souvent employée dans les systèmes d'information classiques.

iv. Variante « max-normalisée »

Elle effectue une normalisation par le terme le plus fréquent du corpus ; utile pour des corpus très déséquilibrés. Le +1 dans le dénominateur évite la division par zéro.

$$IDF_max(t) = \log \frac{\max(df_t)}{1 + df_t}$$

v. Variante utilisée dans BM25

Le modèle BM25 utilise une variante d'IDF légèrement différente, plus agressive pour pénaliser les mots très fréquents :

- Sans lissage :

$$IDF_BM25(t) = \log\left(\frac{N - df_t}{df_t}\right)$$

- Avec lissage (+0,5) pour plus de stabilité numérique :

$$IDF_BM25_smooth(t) = \log\left(\frac{N - df_t + 0.5}{df_t + 0.5}\right)$$

L'ajout de 0,5 empêche les valeurs extrêmes et les valeurs négatives pour $df_t > N/2$.

- Écrire une fonction **calcul_idf(corpus, vocabulaire_direct, methode = 'smooth')** qui calcule l'IDF en appliquant la formule choisie, selon le paramètre « **methode** », pour chaque mot du vocabulaire à partir d'un corpus. Elle retourne un vecteur de valeurs flottantes (IDF) aligné avec les indices du vocabulaire. Le paramètre `methode` ∈ {'classique', 'smooth', 'smooth_P1', 'logplus', 'max', 'bm25', 'bm25_smooth'}.

d. TF-IDF

Le TF mesure l'importance d'un mot dans un document, l'IDF mesure l'importance d'un mot dans le corpus entier. Le TF-IDF combine la fréquence locale (TF) et l'importance globale (IDF) pour pondérer les mots de manière efficace, il valorise les mots fréquents dans un document (TF élevé) mais rares dans les autres documents (IDF élevé) :

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

Exemple :

- "chat dort" $\rightarrow$ TF [0.5, 0, 0.5, 0, 0]
- TF-IDF = [0.5*0.693, 0*0.693, 0.5*0.693, 0*0.693, 0*1.386] $\approx$ [0.347, 0, 0.347, 0, 0]

Document	chat	chien	dort	mange	maison	TF-IDF (vecteur)
"chat dort"	0.347	0	0.347	0	0	[0.347, 0, 0.347, 0, 0]
"chien dort"	0	0.347	0.347	0	0	[0, 0.347, 0.347, 0, 0]
"chat mange"	0.347	0	0	0.347	0	[0.347, 0, 0, 0.347, 0]
"chien mange maison"	0	0.231	0	0.231	0.462	[0, 0.231, 0, 0.231, 0.462]

- Écrire une fonction **calcul_tf_idf(texte, vocabulaire_direct, methode_idf='smooth')** qui transforme un texte en vecteur TF-IDF. Elle retourne une matrice TF-IDF, liste de vecteurs, un par document, où chaque vecteur a la taille du vocabulaire. Où methode_idf $\in$ { 'classique', 'smooth', 'smooth_P1', 'logplus', 'maxidf' }

e. BM25

Le BM25 est une version améliorée du TF-IDF utilisée dans la recherche d'information moderne :

- Il ajuste l'impact de la fréquence des mots et de la longueur du document.
- Paramètres principaux :
 - k_1 : contrôle l'influence de la fréquence du mot, $k_1 \in [1.2, 2.0]$.
 - b : contrôle la normalisation par la longueur du document, $b \in [0.5, 0.8]$.

BM25 produit un score de pertinence pour chaque mot dans un document, qui peut ensuite être utilisé pour représenter un vecteur pondéré pour la recherche d'information.

$$BM25(t, d) = IDF_{BM25_smooth}(t) \times \frac{TF(t, d) \times (k_1 + 1)}{TF(t, d) + k_1 \times (1 - b + b \times \frac{|d|}{longueur\ moyenne})}$$

- Écrire une fonction **calcul_bm25(texte, corpus, vocabulaire_direct, k1=1.5, b=0.75)** qui calcule le vecteur BM25 d'un document. Elle retourne liste de vecteurs, un par document représentant le score BM25 de chaque mot du document

3. Représentations normalisées

Après avoir construit différentes représentations vectorielles (Bag-of-Words, TF, IDF, TF-IDF, BM25), un document reste représenté par un vecteur dont l'échelle dépend fortement de sa longueur ou de ses fréquences internes. Deux documents long et court peuvent donc être difficilement comparables si l'un possède des valeurs beaucoup plus grandes que l'autre.

La normalisation permet de mettre les vecteurs sur une échelle commune, ce qui est indispensable pour :

- comparer des documents entre eux ;
- utiliser des distances (euclidienne, Manhattan, etc.) ;
- appliquer correctement la similarité cosinus ;
- éviter que les documents longs soient avantagés.

Dans cette section, vous allez implémenter les normalisations les plus utilisées en recherche d'information.

a. Normalisation L1 (somme = 1)

La normalisation L1 consiste à diviser chaque composante du vecteur par la somme de toutes les composantes.

$$v_i' = \frac{v_i}{\sum_j v_j} \quad (L1)$$

Cette normalisation transforme le vecteur en une distribution de probabilité. Elle est souvent appliquée aux représentations TF pour obtenir des fréquences relatives.

- Écrire une fonction **normaliser_L1(v)** qui normalise un vecteur selon la norme L1. Elle retourne une liste représentant le vecteur normalisé L1.

b. Normalisation L2 (norme = 1)

La normalisation L2 divise chaque composante par la racine carrée de la somme des carrés.

$$v_i' = \frac{v_i}{\sqrt{\sum_j v_j^2}} \quad (L2)$$

Cette normalisation est la plus importante en recherche documentaire car elle est indispensable au calcul de la similarité cosinus, métrique très utilisée pour comparer des documents.

- Écrire une fonction **normaliser_L2(v)** qui normalise un vecteur selon la norme L2. Elle retourne une liste représentant le vecteur normalisé L2.

c. Normalisation Min–Max (mise à l'échelle entre 0 et 1)

La normalisation Min–Max transforme le vecteur pour que toutes les valeurs soient comprises entre 0 et 1.

$$v_i' = \frac{v_i - \min(v)}{\max(v) - \min(v)}$$

Cette normalisation est surtout utilisée lorsque l'on souhaite comparer des documents dont les caractéristiques doivent être sur la même échelle. Elle est moins courante en recherche documentaire mais importante méthodologiquement.

- Écrire une fonction **normaliser_minmax(v)** qui normalise un vecteur selon la méthode Min-Max. Elle retourne une liste représentant le vecteur normalisé avec Min-Max.

d. Standardisation (z-score)

La standardisation transforme chaque valeur du vecteur en une valeur centrée et réduite. Contrairement aux normalisations L1 et L2 qui modifient l'échelle globale du vecteur, la standardisation s'applique à chaque composante en la ramenant à une échelle commune fondée sur la moyenne et l'écart-type.

$$v_i' = \frac{v_i - \mu}{\sigma}$$

où :

- μ est la moyenne des composantes du vecteur,
- σ est l'écart-type.

Cette méthode est souvent utilisée dans l'apprentissage automatique (SVM, régression, clustering), mais moins courante en recherche d'information.

- Écrire une fonction **standardiser_zscore(v)** qui normalise un vecteur selon la méthode de standardisation. Elle retourne une liste représentant le vecteur standardisé.

4. Représentations basées sur les n-grammes

Jusqu'à présent, toutes les représentations vectorielles reposaient sur un vocabulaire construit à partir de tokens individuels (unigrammes). Cette approche est simple, mais elle ignore complètement l'ordre des mots et les relations locales entre eux. Or, dans de nombreuses tâches de recherche d'information, la présence de séquences de mots (bi-grammes, tri-grammes...) est un indice important de sens ou de structure. Les n-

grammes permettent d'intégrer un minimum de contexte en considérant des séquences contiguës de n tokens.

Lors de la séance précédente, vous avez construit le vocabulaire des n-grammes (`vocabulaire_ngrammes`). Dans cette section, vous allez :

- Construire les dictionnaires d'index direct et inverse pour les n-grammes.
- Créer des représentations vectorielles (BoW, TF, IDF, TF-IDF) basées sur les n-grammes.
- Expérimenter des combinaisons de n-grammes.

a. Construction des dictionnaires d'indexation pour les n-grammes

Chaque n-gramme doit être associé à un indice unique pour encoder les documents.

- Écrire une fonction **`construire_dictionnaire_ngrammes(vocabulaire_ngrammes)`** qui construit deux dictionnaires d'indexation du `vocabulaire_ngrammes` :
 - dictionnaire direct `{ngramme : index}`
 - dictionnaire inverse `{index : ngramme}`

Les indices sont typiquement consécutifs à partir de 0.

b. Encodage vectoriel des n-grammes

Une fois les dictionnaires construits, chaque document peut être représenté par un vecteur de longueur fixe (taille du vocabulaire n-grammes).

Les valeurs du vecteur peuvent être :

- **0/1** : présence/absence (Sac de mots binaire)
- **nombre d'occurrences** (Sac de mots occurrences)
- **TF** : fréquence normalisée dans le document
- **TF-IDF** : pondération par IDF du corpus

Implémenter les fonctions suivantes :

- **`encoder_bow_ngrammes(texte, n, vocab_ng, dico_direct_ng, type='binaire')`**
- **`encoder_tf_ngrammes(texte, n, vocab_ng, dico_direct_ng)`**
- **`encoder_tfidf_ngrammes(texte, n, vocab_ng, dico_direct_ng, idf_ng)`**

L'argument «`type`» permet de choisir la variante `binaire` ou `occurrences`.

c. Combinaison des n-grammes

Pour enrichir la représentation et capturer à la fois la richesse lexicale et le contexte local, il est utile de combiner plusieurs tailles de n-grammes (ex. unigrammes + bigrammes ou unigrammes + bigrammes + trigrammes).

Voici les étapes à suivre :

- i. **Concaténation des vocabulaires** : partir des vocabulaires individuels (vocab_unigrammes, vocab_bigrammes, etc.) et les réunir dans une seule liste.
- ii. **Création des dictionnaires d'indices** :

- dico_direct_final : {ngramme : index}
- dico_inverse_final : {index : ngramme}

Si vos vocabulaires sont correctement construits (unigrammes distincts, bigrammes distincts, etc.), il n'y a pas de doublons entre n-grammes de tailles différentes. La suppression des doublons est donc seulement nécessaire par précaution si des répétitions existent dans un même vocabulaire.

- iii. **Application des encodeurs** : une fois le vocabulaire combiné créé, vous pouvez appliquer les encodeurs existants (BoW, TF, TF-IDF) sur ce vocabulaire final.

- Écrire la fonction `concatener_vocab_ngrammes(liste_vocab_ng)` qui reçoit une liste des vocabulaires de n-grammes à combiner, par exemple [vocab_unigrammes, vocab_bigrammes]. Elle retourne
 - vocab_final : liste des n-grammes combinés, sans doublons
 - dico_direct_final : dictionnaire {ngramme : index}
 - dico_inverse_final : dictionnaire {index : ngramme}